Neuron Activation-based Computation of Logical Explanations for Deep Neural Networks

Tomáš Kolárik[1⋆], Faezeh Labbaf[1], Fabrizio Leopardi[1], Grigory Fedyukovich[1,3], Michael Wand[2], Natasha Sharygina[1]

[1] University of Lugano (USI), Lugano, Switzerland
[2] SUPSI, IDSIA, Lugano, Switzerland
[3] Florida State University, United States of America

Abstract. Formal explainability of classifying neural networks (NNs) is an active area of research, providing explanations with provable guarantees of the classification within continuous regions of the input feature space. However, the existing techniques are either limited to individual input features without guarantees on their relations or the provided solutions fail to scale to deep architectures. This paper addresses these issues by introducing a flexible symbolic framework for an efficient, guided computation of explanations of the NN behavior, parametrized by the activations of internal neurons, and using logical engines such as SMT solvers. Unlike prior methods that rely on specialized NN verifiers, our method yields explanations that are not restricted in shape. Our algorithm is implementable on top of a general-purpose logical solver, isolating the NN-specific encoding from the algorithmic framework. We experimented with a wide range of benchmarks from the domains of image recognition and medicine, illustrating the advantages of the new method, particularly in computational efficiency. Notably, our approach enables logical explanation of deep networks not amenable to prior logic-based methods.

1 Introduction

A recent trend in explaining the behavior of classifying neural networks (NNs) is based on formal methods and first-order logic: Given a NN and a sample point, that is, an assignment of all input features of the NN to concrete values, the problem is to compute a logical justification in the form of a sufficient condition for the NN classification. Logic-based methods encode the NN behavior and the classification outcome into a logical formula and then reason over it while generalizing the constraints on the sample point. Although these methods deliver formal explanations accompanied by *provable guarantees* regarding their correctness and can precisely capture the semantic behavior of the NN within the vicinity of the sample point, their *computation* is a significant *bottleneck* [7,22,23,32,37,38,47,48,51,52], not amenable to realistic deep architectures. Early logic-based methods [7,22,23,37,38,47,48,51,52] rely on symbolic engines that are based on constraint or SMT (Satisfiability Modulo Theories [39,41]) solving, like SMT solvers [5,9–11,13,16] or specialized NN verifiers such as

Marabou [26] to derive explanations via *abductive reasoning*, which originates from program verification [1, 15, 43, 44]. These methods iteratively relax input constraints by enumerating the input features. This process is highly sensitive to feature count, requiring a series of computationally expensive queries to a black-box solver, and depends heavily on selecting a specific feature order. The approach in [7] targets the query overhead by accelerating individual verification calls by abstracting and iteratively refining the original NN, but the sensitivity to feature count remains a bottleneck. In addition to the computational complexity of solving, often similar to an unsatisfiable core extraction, the issue is that the resulting explanations exhibit hyperrectangular shapes that stem from the formula structure and fail to capture input feature relations or to scale beyond specific sample points. Furthermore, the encoding and reasoning over the NN are often coupled: while the symbolic solvers are heavily optimized tools, they could be constrained by such an approach. To address these problems, a new method [29, 32] for computing generic logical explanations that isolates the NN-specific encoding from the reasoning algorithms was recently proposed. The explanations are derived from proofs of unsatisfiability, preserving complex relationships between input features, and utilizing the full power of symbolic solvers without any NN-specific alterations to them. For example, [32] utilizes Craig interpolation [12] and unsatisfiable core extraction to extract formal justifications for classification, removing the need for multiple calls to the underlying solver. While these explanations are correct by construction, not restricted in shape, and offer greater generality than antecedent methods, the approach still scales poorly. Similarly to other prior methods, the problem stems from the direct encoding of the NN without sharing any information about its structure or how it evaluates at the sample point.

To overcome the scalability limitations of all prior logical explanation approaches, we introduce a mechanism that systematically focuses the computation on the sample point while preserving or even improving explanation generality in practice. Our solution decouples and advances both the encoding and solving phases. At the encoding level, we introduce a comprehensive *parametrizable NN encoding* that is driven by the activation of internal neurons at the given sample point, and by an input configuration specific to the NN that is either user-provided or automatically generated by external NN analysis algorithms. The configuration selects internal neurons that are firmly fixed to only one activation phase and neurons where the engine is guided to favor one activation phase over the other. This provides a high flexibility in the trade-off between the computational complexity and the potential size of the space associated with the resulting explanations, while not restricting their shape and remaining provably correct. We also provide an efficient choice of the NN configuration by *identifying important neurons* that maintains a balance in the selection between fixing and guiding the internal nodes based on a gradient traversal from the sample point towards a different class, observing the difference between the neuron activations.

At the solver level, we introduce a novel transferable mechanism to optimize the performance of general-purpose logical solvers such as SAT or SMT

for a concrete application such as NN analysis, by involving *guiding constraints*. Unlike traditional approaches that rely on rigid, general-purpose heuristics, our method uses guiding variables to encode preferred properties into the logical formula, allowing solvers to prioritize decisions systematically aligned with the model's domain-specific structure. The technique can be applied wherever some constraints are more likely to hold than others and in any logical solver handling non-determinism. By leveraging also the order of decision suggestions derived from the NN architecture, the search space is pruned more efficiently than by existing abductive reasoning tools. This contribution bridges the gap between theoretical correctness and practical applicability, offering a scalable solution for generating rigorous logical explanations in deep learning contexts.

Overall, our novel neuron activation-based framework for automated computation of logical explanations is agnostic to the backend solver, as well as to the techniques that generalize the constraints on the sample point, such as Craig interpolation (where various algorithms are available) [32], unsatisfiable core extraction [7,32,51,52], or interval expansion [23,32]. This enables fine-tuning the trade-off between the generality and interpretability of logical explanations—properties discussed in [29]. In particular, when instantiated with Craig interpolation, the resulting explanations are not restricted in shape, unlike those generated by prior methods relying on specialized NN verifiers. Furthermore, the approach generalizes to any acyclic network architecture (e.g., convolutional neural networks—CNNs, ResNets) and activation function, handling arbitrary piecewise-linear functions directly (by guiding or splitting disjunctions) and linearly overapproximating non-piecewise-linear ones [54]. Although beyond the scope of this paper, the computational framework can easily integrate important external components (e.g., [32,33]) to provide various interpretations (e.g., visual) of the logical explanations—an active research field driven by diverse applications such as image recognition and medicine. Notably, while the resulting explanations remain large and complex (sharing the complexity of those in [32]), such interpretability methods render useful visualizations in a matter of seconds.

We implemented the guided approach using the OpenSMT [9,21] solver and experimented with a wide range of real-world benchmarks across a spectrum of complexities from the medical and image-recognition domains, demonstrating the advantages of the new method—particularly regarding its computational efficiency. Notably, our approach enables the explainability of deep networks not amenable to prior logic-based techniques. Compared to the leading abduction-based explainability methods for NNs, namely the case study [32], and the tool VeriX[4] [52], which delivers sound and more compact explanations than other non-logic-based state-of-the-art tools such as Anchors [46] and LIME [45] on the image classification datasets MNIST [14] and GTSRB [50], our experiments show that we not only provide yet more general explanations, but also substantially outperform both [32,52] in these datasets, especially on deep architectures. The

[4] The recently published tool VeriX+ [51] improves the sensitivity of VeriX to the number of input features but still lacks evaluation on deep architectures.

comparative experimentation illustrates the improvement of the generality of the explanations by applying interpretation techniques from prior works [32,33].

As a secondary contribution, we experiment with applying the parametrized NN encoding to an alternative modular framework centered on inductive invariant synthesis. By modeling the network as a discrete-time transition system rather than a monolithic function, we facilitate the discovery of predicates describing relationships among neurons in each layer. Our proof-of-concept experiments demonstrate that leveraging internal neuron configurations significantly enhances formal NN analysis, even when decoupled from traditional monolithic SMT encodings. Overall, the performance improved drastically: when fixing neurons, benchmarks were solved even when they were timing out without the fixing.

Related Work. Logic-based methods formulate the explainability of NNs using abductive reasoning [29]. There is a multitude of such methods [7,22,37,38,47,48], culminating in highly performant VeriX and VeriX+ [51] frameworks. As such, all these techniques are based on specialized NN verifiers and focus only on special cases of the logical concept of abduction, restricting to a specific formula structure or a specific neighborhood of a sample point. As a consequence, they are also sensitive to the number of input features. Similarly, the work in [23] provides interval-based explanations, yet still constrains individual features that cannot reflect feature relationships and hence approximate complex decision boundaries. Overall, these approaches suffer from not being general due to the choice of tools, techniques, and the type of explanations, and the resulting limitations disallow the use of the newly introduced concept of fixing neuron activations that requires the support of unrestricted explanations. The recent case study [32] experimented with using Craig interpolation techniques and unsatisfiable cores to relax input constraints to guarantee the generalization of the explanation. Notably, only one satisfiability query suffices for this purpose. However, the method fails to scale to multi-layer architectures due to direct encoding of the NN. The approach in this paper, while inspired by the generalization ideas of [32], not only elevates the computation of unrestricted logical explanations to a generic symbolic overapproximating reasoning, but more importantly, addresses the scalability issues by parametrizing the explainability task with the NN architecture.

Alongside explainability, NN verification (VNN) is an active field focused on whether a network satisfies a given property (e.g., local robustness). Solving such general satisfiability queries often requires exhaustive exploration of the feature space. The related problems, solutions, and tools are thoroughly surveyed in [25]. Some VNN tools serve as internal engines for early logic-based explainability methods, but their iterative black-box use leads to significant complexity blow-ups. Such tools could be integrated into our neuron activation-based framework if extended to support guiding constraints, but they typically lack generalization techniques (e.g., Craig interpolation), which are crucial for explainability.

Classic (non-logic-based) approaches to NN explainability either perform analysis at the *unit (neuron) level* [17,53], which however fails to capture global properties or interdependencies between units; are *gradient-based* methods [3,49],

which describe the network behavior around a particular sample; or *inversion* methods [36], which provide global, but only approximate, views. There is also a range of *model-agnostic* methods [4,34], including SHAP [30], LIME [45], and Anchors [46], but they often yield logically inconsistent explanations [38]. All the classic methods above rely on approximations with *no strict guarantees* or are *sample-based*: the behavior outside the sampled distribution remains unknown.

Post-processing of logical explanations is an important step for providing a human-understandable interpretation, such as a visualization or formula simplification. There is a whole body of work on interpretation techniques, which is outside the scope of this paper [7,30,32,33,45,46,49,51,52]. The general nature of logical explanations used in this paper makes them amenable to such techniques. In our experiments, we used techniques such as the visualization of the feature space [32], minimum norm point extraction, and hypervolume estimation [33] for comparative analysis.

2 Preliminaries

Neural Network Classifiers. A finite set of input *features* is $\mathcal{F} = \{1, \ldots, m\}$, each associated with a discrete or continuous *domain* $\mathcal{D}_i \subset \mathbb{R}$, either a discrete set of real numbers or a closed interval. A *classifier* is $\mathcal{M} = (\mathcal{F}, \mathbb{F}, \mathcal{K}, \kappa)$, where $\mathbb{F} = \mathcal{D}_1 \times \mathcal{D}_2 \times \cdots \times \mathcal{D}_m$ is the input *feature space*, $\mathcal{K} = \{c_1, c_2, \ldots, c_K\}$ a finite set of K classes, $K \geq 2$, and $\kappa : \mathbb{F} \to \mathcal{K}$ is a *classification function*. An arbitrary point in the feature space is $\mathbf{x} = (x_1, \ldots, x_m) \in \mathbb{F}$, and each $x_i \in \mathcal{D}_i$ is an input *feature variable*. A specific *sample point* in the feature space is $\mathbf{s} = (s_1, \ldots, s_m) \in \mathbb{F}$, where each $s_i \in \mathcal{D}_i$ is a constant. For all classes, at least one sample point must be classified into the class. A (fully connected) *classifying neural network* (NN) is a classifier represented by a graph $(\mathcal{V}, \mathcal{E})$ with the set of vertices $\mathcal{V}$ and the set of edges $\mathcal{E}$, representing the neurons and their weighted connections, respectively. The approach in this paper also extends to other acyclic architectures (e.g., CNNs). Neurons $v_i^{(\ell)}$ are partitioned into $L+1$ layers $\mathcal{V}^{(0)}, \ldots, \mathcal{V}^{(L)}$ such that each $\mathcal{V}^{(\ell)} = \{v_i^{(\ell)}\}_{i=1}^{n^{(\ell)}}$ contains $n^{(\ell)}$ neurons. Layer $\mathcal{V}^{(0)}$ is the *input* layer, $\mathcal{V}^{(L)}$ is the *output* layer, layers $\mathcal{V}^{(1)}, \ldots, \mathcal{V}^{(L-1)}$ are *hidden* layers, and $\mathcal{V}_{\mathcal{H}} = \bigcup_{\ell=1}^{L-1} \mathcal{V}^{(\ell)}$ is the set of neurons in hidden layers. For each $0 \leq \ell < L$, every neuron $v_i^{(\ell)} \in \mathcal{V}^{(\ell)}$ is connected to every neuron $v_j^{(\ell+1)} \in \mathcal{V}^{(\ell+1)}$ by an edge $(v_i^{(\ell)}, v_j^{(\ell+1)}) \in \mathcal{E}$, which is assigned a weight $w_{i,j}^{(\ell)} \in \mathbb{R}$, and for each $1 \leq \ell \leq L$, every neuron $v_i^{(\ell)}$ is assigned a bias $b_i^{(\ell)} \in \mathbb{R}$. Each input neuron $v_i^{(0)}$ is associated with input feature variable $x_i \in \mathcal{D}_i$ and each output neuron $v_i^{(L)}$ to class $c_i \in \mathcal{K}$. Given a neuron $v_i^{(\ell)}$, $z_i^{(\ell)} : \mathbb{F} \to \mathbb{R}$ is the *pre-activation* and $a_i^{(\ell)} : \mathbb{F} \to \mathbb{R}$ is the *activation*, defined as follows: $z_i^{(0)}(\mathbf{x}) = a_i^{(0)}(\mathbf{x}) = x_i$, and for hidden and output layers ($1 \leq \ell \leq L$):

$$z_i^{(\ell)}(\mathbf{x}) = \sum_{j=1}^{n^{(\ell-1)}} w_{j,i}^{(\ell-1)} a_j^{(\ell-1)}(\mathbf{x}) + b_i^{(\ell)} \tag{1}$$

$$a_i^{(\ell)}(\mathbf{x}) = \sigma\big(z_i^{(\ell)}(\mathbf{x})\big) \tag{2}$$

where $\sigma(z) = ReLU(z) = \max\{z, 0\}$ in the case of hidden layers and $\sigma(z) = z$ in the case of output layers. A neuron $v_i^{(\ell)} \in \mathcal{V}_{\mathcal{H}}$ is *active* w.r.t. input $\mathbf{x} \in \mathbb{F}$ if $z_i^{(\ell)}(\mathbf{x}) > 0$, and *inactive* if $z_i^{(\ell)}(\mathbf{x}) \leq 0$. This can be extended to arbitrary piecewise-linear σ or an overapproximation [54]. The classification is given by the output neuron with the maximum activation: $\kappa(\mathbf{x}) = c_i \iff i = \text{argmax}_j\{a_j^{(L)}\}$.

SMT Encoding of Neural Networks. Using an SMT-LIB [6] quantifier-free first-order logic, given a class $c \in \mathcal{K}$, $\kappa(\mathbf{x}) \neq c$ is encoded by $\psi := \psi_{\mathcal{M}} \wedge \psi_{\mathcal{D}} \wedge \neg\psi_c$:

$\psi_{\mathcal{M}}$: The neural network $\mathcal{M}$: the constraints in (1), with fixed weights and biases, and in (2), are encoded directly. The $ReLU$ function can be encoded using the if-then-else (ITE) term of the SMT-LIB standard.

$\psi_{\mathcal{D}}$: All domains $\mathcal{D}_i$ of the feature space, that is, $\mathbf{x} \in \mathbb{F}$ (i.e., $x_i \in \mathcal{D}_i$ for all $i \in \mathcal{F}$).

$\neg\psi_c$: The constraint on the outcome of the classification is *not* class c: the value of at least one output neuron $a_i^{(L)}(\mathbf{x})$ exceeds that of class c.

Functions $z_i^{(\ell)}$ and $a_i^{(\ell)}$ are encoded via inlining or auxiliary variables. Since $K \geq 2$ and classes are not redundant, ψ is satisfiable. Given a sample point $\mathbf{s} \in \mathbb{F}$ s.t. $\kappa(\mathbf{s}) = c$, and using $\varphi_{\mathbf{s}} := \bigwedge_{i \in \mathcal{F}} x_i = s_i$, formula $\varphi_{\mathbf{s}} \wedge \psi$ is *unsatisfiable*. We instantiate the SMT-LIB logic to QF_LRA (Quantifier-Free Linear Real Arithmetic). Alternative encodings like QF_FP use floating-point semantics, but sound solving and interpolation are much more challenging in such settings.

Logical Explanations. Our solution aims at general logic-based explanations. Thus, we define a general logical concept using the notion of abductive reasoning[5] that subsumes all prior abduction-based concepts [7,22,23,37,38,47,48,51,52][6].

Definition 1 (Logical Explanation, Impact Space). *Given a classifier* $\mathcal{M} = (\mathcal{F}, \mathbb{F}, \mathcal{K}, \kappa)$*, and a class* $c \in \mathcal{K}$*, a* logical explanation *of class* c *is a satisfiable formula* φ *such that* $\forall\, \mathbf{x} \in \mathbb{F} \,.\, \varphi(\mathbf{x}) \implies \kappa(\mathbf{x}) = c$*. The* impact space *of* φ *is the set* $\mathbb{F}_\varphi := \{\mathbf{x} \in \mathbb{F} \mid \varphi(\mathbf{x})\} \subset \mathbb{F}$*.*

Logical explanations with the smallest impact space correspond to a sample point $\mathbf{s} \in \mathbb{F}$: $\varphi_{\mathbf{s}}$ with $\mathbb{F}_{\varphi_{\mathbf{s}}} = \{\mathbf{s}\}$. Logical explanations are sufficient conditions of classification to class c: they guarantee the classification for all points covered by the impact space. Unlike many prior methods, the concept can be applied both to global and local[7] explanations since it does not rely on a sample point. The general logical representation of explanations is inherently flexible, making it applicable to a wide range of interpretation techniques—an active research area on its own (e.g., [32,33,52]) driven by diverse, application-specific user needs.

[5] For vectors of variables $\mathbf{x}, \mathbf{y}$ and formulas $T(\mathbf{x}, \mathbf{y}), O(\mathbf{x}, \mathbf{y})$, the *abduction problem* is to find a formula $\varphi(\mathbf{x})$ s.t.: (1) $\varphi(\mathbf{x}) \wedge T(\mathbf{x}, \mathbf{y}) \not\Rightarrow \bot$, and (2) $\varphi(\mathbf{x}) \wedge T(\mathbf{x}, \mathbf{y}) \Rightarrow O(\mathbf{x}, \mathbf{y})$.

[6] Our generalization is influenced by the recent case study [32] on the application of Craig interpolation for the explainability of classifying NNs. Theoretical comparison of [32] with other abduction-based concepts is available in [29].

[7] Given a sample $\mathbf{s}$ (can also be randomly selected), φ s.t. $\mathbf{s} \in \mathbb{F}_\varphi$ is a local explanation.

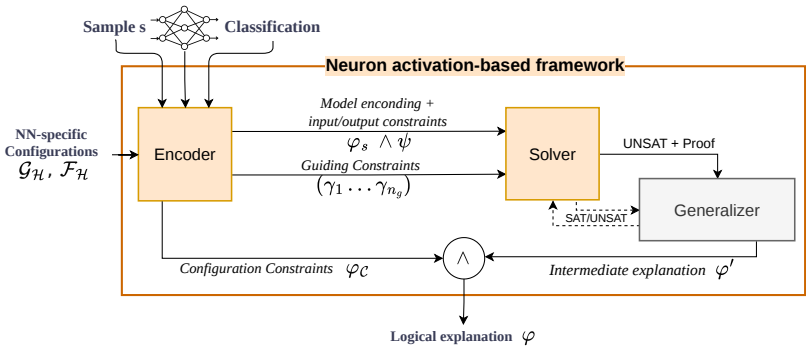


Fig. 1: Neuron activation-based framework for automated computation of logical explanations, which takes as input a sample point, NN, the classification, and configurations of internal neurons, and outputs a logical explanation.

Various symbolic techniques for the construction of generalized logical explanations can be employed, such as unsatisfiable core extraction, interval expansion, or Craig interpolation[8]. A Craig interpolant $I = Itp(\varphi, \psi)$, where φ is a logical explanation and ψ encodes $\kappa(\mathbf{x}) \neq c$, is a logical explanation of c satisfying $\varphi \Rightarrow I$ (hence $\mathbb{F}_{\varphi} \subseteq \mathbb{F}_I$). Various interpolation algorithms exhibit different logical strengths of the produced formulas [8]: Farkas' lemma (Itp_F), its logically stronger variant (Itp_{DF}), their dual versions (Itp'_F, Itp'_{DF}), and an algorithm with a flexible strength (Itp_f) [2] parametrized by a rational factor $f \in [0, 1]$. The explanations generated by these algorithms, when applied to the same arguments, follow $Itp_{DF} \Rightarrow Itp_F \Rightarrow Itp_f \Rightarrow Itp'_F \Rightarrow Itp'_{DF}$.

The next sections describe the contributions of this paper, which enable efficient computation of generic logical explanations. The generalization is not restricted to any particular method and is suitable for a wide range of solvers. Sect. 3 describes particular techniques for domain-specific guidance within a parametrized encoding of NNs, and Sect. 4 addresses a generic but powerful idea of guiding constraints that aim at general-purpose logical solvers.

3 Neuron Activation-based Logical Explanations

This section introduces a flexible symbolic framework for an efficient, guided computation of logical explanations of the NN behavior, parametrized by the activations of internal neurons, and using logical solvers such as SMT solvers. In particular, it focuses on a comprehensive parametrizable encoder of the NN classifier—namely, the constraints on weights and biases (1) and neuron outputs (2)—which is driven by the activation of internal neurons at the given sample point being explained, and by an input configuration specific to the NN.

[8] Given an unsatisfiable formula $A \wedge B$, a *Craig interpolant* is a formula I s.t.: $A \Rightarrow I$, $I \wedge B$ is unsatisfiable, and I uses only the common variables of A and B. $I = Itp(A, B)$ denotes the interpolant I computed by an interpolation algorithm Itp from A and B.

Notably, as opposed to many prior approaches, the encoder is isolated from the symbolic computational engine, which is addressed in the next section.

Fig. 1 gives an overview of our framework for computing logical explanations with a parametrized encoding of the input neural network, highlighting in orange the contributions of this paper. The **Encoder** processes a given neural network, input sample, and its classification, yielding a logical formula $\varphi_{\mathbf{s}} \wedge \psi$ that is unsatisfiable by construction (cf. Sect. 2). Notably, the **Encoder** is parametrizable by NN-specific configurations $\mathcal{G}_{\mathcal{H}}$ and $\mathcal{F}_{\mathcal{H}}$ that define specific internal neurons whose activation should be guided ($\mathcal{G}_{\mathcal{H}}$) or fixed ($\mathcal{F}_{\mathcal{H}}$), optionally producing additional constraints: $\mathcal{G}_{\mathcal{H}}$ controls solely *guiding constraints* $(\gamma_1, \ldots, \gamma_{n_g})$, while $\mathcal{F}_{\mathcal{H}}$ may restrict the encoding ψ as well as produce additional *configuration constraints* $\varphi_{\mathcal{C}}$ to maintain the soundness of the resulting explanation φ. This parametrized encoding helps the logical **Solver**, such as a proof-producing SMT solver, to prove the unsatisfiability of $\varphi_{\mathbf{s}} \wedge \psi$ faster: one of the branches in each $ReLU$ activation defined in (2) within the encoding ψ may be firmly pruned (via $\mathcal{F}_{\mathcal{H}}$) or preferred over the other (via $\mathcal{G}_{\mathcal{H}}$). This becomes especially important for deep NNs with many hidden layers (i.e., with many consecutive $ReLU$ case splits). To allow the guidance of the reasoning over ψ via $\mathcal{G}_{\mathcal{H}}$, the **Solver** implements a new concept of *decision guidance* (see Sect. 4), given the additional ordered input $(\gamma_1, \ldots, \gamma_{n_g})$ generated based on $\mathcal{G}_{\mathcal{H}}$, where each constraint γ_i (here an activation of an internal neuron) is favored to be satisfied rather than not. Then, the **Solver** extracts a proof of the unsatisfiability of $\varphi_{\mathbf{s}} \wedge \psi$, which is subsequently applied to the **Generalizer** that generalizes formula $\varphi_{\mathbf{s}}$ into an intermediate explanation φ'. Combined with the logical encoding of NN configuration constraints $\varphi_{\mathcal{C}}$, the final explanation results in $\varphi = \varphi' \wedge \varphi_{\mathcal{C}}$. The **Generalizer** can be implemented in various ways as long as the overapproximation of formula $\varphi_{\mathbf{s}}$ is computed and thus directly produces a φ'. For example, Craig interpolation procedures can be used to extract φ' as a computed interpolant. Another option is extracting an unsatisfiable core—removing parts of $\varphi_{\mathbf{s}}$ while maintaining its unsatisfiability—but requiring additional calls (SAT/UNSAT) to the **Solver**. The resulting logical explanations are amenable to further interpretations (e.g., [32, 33]).

The rest of the section describes how the **Encoder** forms its outputs (**Solver** inputs, cf. Sect. 4), given NN configurations $\mathcal{G}_{\mathcal{H}}$ and $\mathcal{F}_{\mathcal{H}}$: model encoding $\psi_{\mathcal{M}}$ (a part of ψ), guiding constraints $(\gamma_1, \ldots, \gamma_{n_g})$, and configuration constraints $\varphi_{\mathcal{C}}$.

Decision Guidance via Neuron Activations. Logical solvers can handle focused, structure-aware guiding constraints (cf. Sect. 4), which we exploit to support direct NN encoding (cf. Sect. 2) by suggesting preferred internal neuron activations. In Fig. 1, direct NN encoding corresponds to model encoding ψ, sent alongside guiding constraints $(\gamma_1, \ldots, \gamma_{n_g})$ to the **Solver**. Alg. 1 describes how to encode neural network $\mathcal{M}$ and a sample point $\mathbf{s} \in \mathbb{F}$ into guiding constraints, driven by a set $\mathcal{G}_{\mathcal{H}}$ (a part of NN configurations in Fig. 1) specifying which neurons to guide. The choice of the set $\mathcal{G}_{\mathcal{H}}$ is specific to the network $\mathcal{M}$ and can be user-provided or automatically generated by external NN analysis

Algorithm 1: Generating neuron activation-based guiding constraints

Input: $\mathcal{M} = (\mathcal{F}, \mathbb{F}, \mathcal{K}, \kappa)$, sample point $\mathbf{s} \in \mathbb{F}$, set of neurons to guide $\mathcal{G}_\mathcal{H} \subseteq \mathcal{V}_\mathcal{H}$
Output: ordered set of n_g guiding constraints $(\gamma_1, \gamma_2, \ldots, \gamma_{n_g})$, $n_g = |\mathcal{G}_\mathcal{H}|$

▷ following Sect. 2, each $z_i^{(\ell)}$ refers to (1), $v_i^{(\ell)}$ to a neuron,
▷ ... L to the number of layers, and $n^{(\ell)}$ to the number of neurons in layer ℓ
$\mathbf{\Gamma} \leftarrow ()$ ▷ initialize the list of guiding constraints
for $\ell \leftarrow 1;\ \ell \leq L-1;\ \ell \leftarrow \ell+1$ **do** ▷ iterate over all hidden layers
  for $i \leftarrow 1;\ i \leq n^{(\ell)};\ i \leftarrow i+1$ **do** ▷ iterate over all neurons of layer ℓ
    ▷ guide solver to prefer the (in)active phase for (in)active neurons from $\mathcal{G}_\mathcal{H}$
    if $v_i^{(\ell)} \in \mathcal{G}_\mathcal{H}$ **then** $\mathbf{\Gamma} \leftarrow \mathbf{\Gamma} \parallel \big(z_i^{(\ell)}(\mathbf{s}) > 0\big)\ ?\ \big(z_i^{(\ell)} > 0\big) : \big(z_i^{(\ell)} \leq 0\big)$
return $\mathbf{\Gamma}$

algorithms. The algorithm strictly follows the order of hidden layers and neurons (though other orders are possible). The guiding constraints focus on the pre-activation (1) of neurons $z_i^{(\ell)}$ evaluated at $\mathbf{s}$, *guiding* the solver toward the desired activation (2), either $a_i^{(\ell)} = z_i^{(\ell)}$ or $a_i^{(\ell)} = 0$, within the ITE encoding. For example, consider a NN with $\mathbf{x} = (x_1, x_2, x_3)$ and a neuron $v_i^{(1)} \in \mathcal{G}_\mathcal{H}$ (in the first hidden layer) with pre-activation $z_i^{(1)} = 2a_1^{(0)} + 0a_2^{(0)} + a_3^{(0)} = 2x_1 + x_3$. Given sample $\mathbf{s}_1 = (1, 1, 3)$, Alg. 1 adds guiding constraint $z_i^{(1)} > 0$ to $\mathbf{\Gamma}$; given sample $\mathbf{s}_2 = (0, 2, 0)$, it adds $z_i^{(1)} \leq 0$. Notably, guiding constraints affect neither satisfiability soundness nor explanation correctness. Moreover, the order of neurons and of their activations is not strictly enforced, as the solver considers them solely during the decisions, ignoring them during conflict analysis and propagation. See Sect. 4 for more details.

A direct strategy for Alg. 1 is to set $\mathcal{G}_\mathcal{H} = \mathcal{V}_\mathcal{H}$, guiding the decisions on all neuron activations in the network. This is in fact usually the optimal choice since using guiding constraints improves both performance and generality in practice.

NN Encoding Parametrizable by Fixing Neuron Activations. This part describes how to encode a neural network $\mathcal{M}$ into formula $\psi_\mathcal{M}$ (cf. Sect. 2) while optionally fixing selected neurons to one activation phase—that is, selecting one branch of the *ReLU* splits (2). In Fig. 1, $\psi_\mathcal{M}$ is a conjunct of the model encoding ψ. If neurons are fixed, $\psi_\mathcal{M}$ encodes a stricter variant of $\mathcal{M}$. To ensure that the resulting explanation φ is valid w.r.t. $\mathcal{M}$, the **Encoder** also produces configuration constraints $\varphi_\mathcal{C}$: in Fig. 1, these are applied to the intermediate explanation φ' produced by the **Generalizer** such that $\varphi := \varphi' \wedge \varphi_\mathcal{C}$. Then, φ constrains parts of the feature space where only the fixed activation phase of neurons is reachable. This technique is *orthogonal* to the decision guidance above, which addresses only optional neuron activation guiding constraints. Alg. 2 describes the process of encoding $\psi_\mathcal{M}$ and configuration constraints $\varphi_\mathcal{C}$, given $\mathcal{M}$, a sample point $\mathbf{s}$, and a set of neurons $\mathcal{F}_\mathcal{H}$ to be fixed. The set $\mathcal{F}_\mathcal{H}$ is an input similar to $\mathcal{G}_\mathcal{H}$ in Alg. 1, but forms a *separate* part of the NN configurations in Fig. 1. Alg. 2 encodes the constraints depending on the input feature variables $\mathbf{x} = (x_1, \ldots, x_m)$ and again strictly follows the order of layers and neurons. Direct encoding using

Algorithm 2: Parametrizable NN encoding, fixing neuron activations

Input: $\mathcal{M} = (\mathcal{F}, \mathbb{F}, \mathcal{K}, \kappa)$, sample point $\mathbf{s} \in \mathbb{F}$, set of neurons to fix $\mathcal{F}_\mathcal{H} \subseteq \mathcal{V}_\mathcal{H}$
Output: NN encoding $\psi_\mathcal{M}$, configuration constraints $\varphi_\mathcal{C}$

▷ notation as in Alg. 1; $a_i^{(\ell)}$ refers to (2); $a_i^{(0)} = x_i$ (input feature variables)
$\psi_\mathcal{M} \leftarrow \top, \varphi_\mathcal{C} \leftarrow \top$ ▷ initialize encodings of $\mathcal{M}$ and the configuration constraints
for $\ell \leftarrow 1;\ \ell \leq L-1;\ \ell \leftarrow \ell+1$ **do** ▷ iterate over all hidden layers
  for $i \leftarrow 1;\ i \leq n^{(\ell)};\ i \leftarrow i+1$ **do** ▷ iterate over all neurons of layer ℓ
    if $v_i^{(\ell)} \notin \mathcal{F}_\mathcal{H}$ **then** ▷ do not fix neurons that are not included in $\mathcal{F}_\mathcal{H}$
      $\psi_\mathcal{M} \leftarrow \psi_\mathcal{M} \wedge \left(a_i^{(\ell)} = ite\big(z_i^{(\ell)} > 0, z_i^{(\ell)}, 0\big)\right)$ ▷ direct ITE encoding
    else
      $\psi_\mathcal{M} \leftarrow \psi_\mathcal{M} \wedge \big(z_i^{(\ell)}(\mathbf{s}) > 0\big)\ ?\ \big(a_i^{(\ell)} = z_i^{(\ell)}\big) : \big(a_i^{(\ell)} = 0\big)$ ▷ fix the activation
      $\varphi_\mathcal{C} \leftarrow \varphi_\mathcal{C} \wedge \big(z_i^{(\ell)}(\mathbf{s}) > 0\big)\ ?\ \big(z_i^{(\ell)} > 0\big) : \big(z_i^{(\ell)} \leq 0\big)$ ▷ constrain the activation
$\psi_\mathcal{M} \leftarrow \psi_\mathcal{M} \wedge \bigwedge_{i=1}^{n^{(L)}} (a_i^{(L)} = z_i^{(L)})$ ▷ no $ReLU$ in the output layer
return $\psi_\mathcal{M}$, $\varphi_\mathcal{C}$

an ITE term is applied for neurons not specified in $\mathcal{F}_\mathcal{H}$; otherwise, only one of the branches is encoded, depending on the evaluation of the pre-activation value $z_i^{(\ell)}$ at the sample point $\mathbf{s}$. The algorithm also stores the pre-activation constraint into configuration constraints $\varphi_\mathcal{C}$ to maintain the restriction and soundness of the resulting explanations. For example, consider the same NN and neuron $v_i^{(1)}$. If $v_i^{(1)} \notin \mathcal{F}_\mathcal{H}$, its activation is encoded as $a_i^{(1)} = ite(z_i^{(1)} > 0, z_i^{(1)}, 0)$ as part of $\psi_\mathcal{M}$. If $v_i^{(1)} \in \mathcal{F}_\mathcal{H}$, given sample $\mathbf{s}_1 = (1, 1, 3)$, $a_i^{(1)} = z_i^{(1)}$; given sample $\mathbf{s}_2 = (0, 2, 0)$, $a_i^{(1)} = 0$, while adding $z_i^{(1)} > 0$ and $z_i^{(1)} \leq 0$ to $\varphi_\mathcal{C}$, respectively. At the end of Alg. 2, it encodes output neuron constraints where no $ReLU$ function is applied. Notably, such a φ is a correct logical explanation w.r.t. $\mathcal{M}$ (Definition 1) since the form of the formula is not restricted.

Theorem 1. *Given a neural network $\mathcal{M}$ and a sample point $\mathbf{s} \in \mathbb{F}$ classified as class c by $\mathcal{M}$, let $\psi'_\mathcal{M}$ and $\varphi_\mathcal{C}$ be the formulas produced by Alg. 2, $\mathcal{M}'$ be the restricted NN encoded by $\psi'_\mathcal{M}$, and φ' be a logical explanation of $\mathcal{M}'$ and c such that $\varphi_\mathbf{s} \implies \varphi'$. Then, $\varphi := \varphi' \wedge \varphi_\mathcal{C}$ is a logical explanation of $\mathcal{M}$ and class c.*

Proof. Let $\psi_\mathcal{M}$ be the encoding of the original network $\mathcal{M}$, then $\psi := \psi_\mathcal{M} \wedge \psi_\mathcal{D} \wedge \neg\psi_c$ encodes the change of classification c in $\mathcal{M}$. We need to prove that φ is satisfiable and $\varphi \wedge \psi$ is unsatisfiable. The satisfiability of φ (i.e., $\varphi' \wedge \varphi_\mathcal{C}$) follows since $\varphi_\mathbf{s} \wedge \varphi_\mathcal{C}$ is satisfiable ($\varphi_\mathcal{C}$ is constructed based on $\mathbf{s}$) and the premise $\varphi_\mathbf{s} \Rightarrow \varphi'$. Let $\psi' := \psi'_\mathcal{M} \wedge \psi_\mathcal{D} \wedge \neg\psi_c$. Since $\varphi' \wedge \psi'$ is unsatisfiable, so is $\varphi' \wedge \psi \wedge \varphi_\mathcal{C}$ because $\psi_\mathcal{M} \wedge \varphi_\mathcal{C} \equiv \psi'_\mathcal{M}$ and hence $\psi \wedge \varphi_\mathcal{C} \equiv \psi'$. Therefore, $\varphi \wedge \psi$ is also unsatisfiable. □

This universally produces logical explanations from sample points. For instance, given $\mathcal{M}$, sample $\mathbf{s}$ classified as c, $\psi_\mathcal{M}$ and $\varphi_\mathcal{C}$ produced by Alg. 2, and using an interpolant $\varphi' = Itp(\varphi_\mathbf{s}, \psi_\mathcal{M} \wedge \psi_\mathcal{D} \wedge \neg\psi_c)$, where Itp is a Craig interpolator, then $\varphi := \varphi' \wedge \varphi_\mathcal{C}$ is a logical explanation of $\mathcal{M}$ and class c such that $\varphi_\mathbf{s} \Rightarrow \varphi$.

When the activation phase of a neuron at sample $\mathbf{s}$ is *unstable*—i.e., pre-activation $z_i^{(\ell)}(\mathbf{s})$ in Alg. 2 is close to zero—fixing such a neuron may be too

restrictive, potentially resulting in a non-robust explanation. This can be avoided by extending the condition in line 4 with a disjunction for $|z_i^{(\ell)}(\mathbf{s})| \leq \varepsilon$, using a small ε (e.g., $\varepsilon = 10^{-3}$ for normalized domains $\mathcal{D}_i$) to enforce direct encoding.

A straightforward strategy is to set $\mathcal{G}_\mathcal{H} = \emptyset$ for Alg. 1 and $\mathcal{F}_\mathcal{H} = \mathcal{V}_\mathcal{H}$ for Alg. 2, which removes all branchings from $\psi_\mathcal{M}$ (if no activations are unstable). Even then, ψ may not be deterministic, due to classification constraints ψ_c (cf. Sect. 2). The combination of Alg. 2 and Alg. 1 can be optimized not to produce guiding constraints for neurons that have been fixed. Hence, it still makes sense to apply the input sets with $\mathcal{F}_\mathcal{H} \cap \mathcal{G}_\mathcal{H} \neq \emptyset$ (neurons with unstable activation are not fixed), and a meaningful combination is even $\mathcal{F}_\mathcal{H} = \mathcal{G}_\mathcal{H} = \mathcal{V}_\mathcal{H}$. However, if fixing all neurons is overly restrictive, it is useful to identify $\mathcal{F}_\mathcal{H}$ as a subset of $\mathcal{V}_\mathcal{H}$. The following paragraph introduces an algorithm to identify relevant neurons, which is useful for finding an efficient combination of configurations $\mathcal{F}_\mathcal{H}$ and $\mathcal{G}_\mathcal{H}$.

Finding Relevant Neurons by Crossing the Decision Boundary. Relevant neurons $\mathcal{R}_\mathcal{H}$ in hidden layers can be identified given two sample points $\mathbf{s}, \mathbf{s}' \in \mathbb{F}$ classified into different classes by the network (i.e., $\kappa(\mathbf{s}) \neq \kappa(\mathbf{s}')$):

$$\mathcal{R}_\mathcal{H} = \left\{ v_i^{(\ell)} \mid \ell \in [1, L), i \in [1, n^{(\ell)}], (a_i^{(\ell)}(\mathbf{s}) = 0) \neq (a_i^{(\ell)}(\mathbf{s}') = 0) \right\}. \tag{3}$$

The **Encoder** leaves these neurons *unfixed*. By (3), their activation phase changes when evaluated at $\mathbf{s}$ and $\mathbf{s}'$ across the decision boundary. This preserves the flexibility needed to capture feature relations associated with this boundary crossing. All other neurons are fixed ($\mathcal{F}_\mathcal{H} = \mathcal{V}_\mathcal{H} \setminus \mathcal{R}_\mathcal{H}$), with $\mathcal{G}_\mathcal{H} = \mathcal{V}_\mathcal{H}$ as the fallback.

To restrict the state space and maximize the benefits of fixing, $\mathcal{R}_\mathcal{H}$ must be kept small. For this reason, $\mathbf{s}'$ is computed as an *adversarial example* [20] of sample $\mathbf{s}$, by adding a small, carefully chosen perturbation to $\mathbf{s}$ that changes the classification. In the literature, adversarial examples are typically constructed by solving a constrained optimization problem over a small perturbation to maximize the loss of the original class, often via iterative search within a neighborhood of the original sample $\mathbf{s}$. A popular technique for this purpose is *Projected Gradient Descent* (PGD) [35]. Starting from an initial point close to $\mathbf{s}$, using a fixed number of iterations, PGD updates the position in a direction that increases the loss of the classifier. After each update, the point is projected back into a small vicinity of $\mathbf{s}$, bounding the perturbation and the resulting adversarial example.

Overall, each presented technique contributes to the performance of computing NN explanations and can be combined on demand, as illustrated in Sect. 5.

4 Decision Guidance of Logical Solvers

Logic-based explainability methods rely on efficient symbolic logical solvers, such as SAT, SMT, or MILP, as underlying computational engines. These solvers are highly optimized for general constraint solving. However, generic heuristics may not be optimal for specific domain-driven problems. This paper proposes guiding constraints as a transferable mechanism to optimize the performance of logical solvers for NN analysis. A *guiding constraint* is a formula defining a

Algorithm 3: Check the satisfiability with propositional decision guidance

Input: formula ϕ, finite ordered set of n_g guiding constraints $(\gamma_1, \gamma_2, \ldots, \gamma_{n_g})$
Output: `sat` or `unsat` if ϕ is satisfiable or not, resp.

```
1  (g_1, g_2, ..., g_{n_g}) ← MakeNewVariables(n_g)      ▷ n_g fresh propositional variables
2  φ ← φ ∧ ⋀_{i=1}^{n_g} (g_i ⟹ γ_i)                     ▷ update the formula
   ▷ get a propositional assignment A
3  A ← CheckSat*(φ, (g_1, g_2, ..., g_{n_g}))      ▷ calls SelectDecisionLiteral* inside
4  if A satisfies φ and contains all propositional variables then      ▷ full assignment
5  |  return sat
6  return unsat
7  function SelectDecisionLiteral*(A, (g_1, g_2, ..., g_{n_g}))
8  |  for i ← 1; i ≤ n_g; i ← i + 1 do                  ▷ strictly follow the order
9  |  |  if g_i ∉ A then return g_i                     ▷ select literal g_i if not assigned yet
10 |  return SelectInternalDecisionLiteral(A)           ▷ use internal heuristics
```

preferred property rather than a mandatory requirement, which can be a natural, yet optional, part of the input. The solver implementation is still allowed to completely ignore it. Given a formula ϕ and a guiding constraint γ, we introduce a *fresh* propositional variable g, called the *guiding variable* of γ. Notably, γ is not limited to existing variables, literals, or even terms in ϕ. Then, because ϕ is equisatisfiable with $\phi \wedge (g \Rightarrow \gamma)$, a solver can enforce γ via a propositional decision on g. Internally, the solver constructs a sequence of guiding variables g_i that serve as systematic decision suggestions for logical constraints γ_i. Crucially, the *order* of decision suggestions *matters*: many models naturally follow a domain-specific topology that is entirely lost in standard logical encodings focused solely on firm constraints. Moreover, extracting an effective decision order from structured models, such as a NN aligned into layers, is straightforward.

Alg. 3 describes the main idea of the decision guidance via propositional decisions within a logical solver checking the satisfiability of a formula ϕ, given an *ordered* set of guiding constraints. Although the algorithm is straightforward and introduces only a few solver modifications (highlighted in blue), implementation requires internal knowledge of the target tool to integrate the feature into the decision heuristics, often within a highly optimized environment. In lines 1–2, the algorithm creates guiding variables and updates formula ϕ. In line 3, it calls CheckSat*, which differs from standard solving by using SelectDecisionLiteral* instead of SelectInternalDecisionLiteral for decision choices of propositional literals. In SelectDecisionLiteral* (lines 8–9), the solver first attempts to pick an unassigned guiding variable, selecting the positive literal g_i as the decision literal. As long as some g_i remains unassigned, decision guidance takes precedence over default internal heuristics. Notably, SelectDecisionLiteral* affects only decision choices, leaving conflict reasoning and propagation untouched. Hence, any g_i that conflicts with the current assignment is skipped ($g_i \notin \mathbf{A}$ in line 9).

When γ_i is itself a propositional literal or has an existing propositional abstraction in the solver, the implementation can omit creating g_i and use that

literal directly. While Alg. 3 targets DPLL-based SMT solvers, frameworks supporting non-propositional decisions (e.g., MCSat [40]) can implement decision guidance without explicit guiding variables.

A restricted form of this capability was previously introduced in MathSAT5 [11], requiring guiding constraints to be propositional literals in the input formula ϕ. The novel, unrestricted feature was first implemented in Z3 [13] and OpenSMT, where guiding the computation of NN explanations yielded orders-of-magnitude speedups over unguided reasoning—a gain transferable to any supporting SMT solver.

This general concept applies to diverse logical solvers and domains. In this paper, we leverage decision suggestions to guide the NN explanation process toward specific neuron activations, aligned with a sample point and the topological order of a network. Concretely, this instantiation appears in Fig. 1 within the **Solver** implementing Alg. 3, accepting guiding constraints $(\gamma_1, \ldots, \gamma_{n_g})$ from the **Encoder** as input. This application utilizes a subset of the full capability: individual constraints γ_i (neuron activations) are already present in $\psi_{\mathcal{M}}$, though Alg. 3 supports arbitrary fresh constraints.

5 Experimental Evaluation

We implemented the neuron activation-based approach using a parametrized encoding of the NN w.r.t. a specific configuration of the internal neurons within the **Encoder** (Alg. 1 and 2)[9]. Computation of configurations that are based on (3) is implemented using TorchAttack library [27] for PGD. We stress-tested the performance of our techniques on the classical image-classification datasets, traditionally used to study the performance limits of logical explanation techniques for classification: GTSRB (3072 inputs, 43 classes), CIFAR10 [31] (3072 inputs, 10 classes), and MNIST (784 inputs, 10 classes). We also used two smaller tabular medical domain benchmarks that operate on very concrete domain inputs and are used by medical practitioners: heart-attack risk [24] (13 inputs, 2 classes), and obesity [42] (15 inputs, 7 classes) datasets. Here, not only the individual features but also the relationships among them are highly relevant. For each dataset, we randomly selected 100 sample points, and trained fully connected NNs with 1 to 14 layers for the image-classification datasets and with 1 to 8 layers for the medical datasets[10], each with 50 neurons per hidden layer. This paper focuses primarily on the monolithic approach that is based on SMT, evaluated in Sect. 5.1. Additionally, in Sect. 5.2, we evaluate an alternative experimental modular variant to observe how internal neuron activation configurations influence non-interpolation-based NN analysis. Although we omit the formal details of the modular encoding for brevity, we provide the underlying intuition, noting that specific optimizations from Alg. 2 can be effectively migrated to this setting.

[9] https://github.com/usi-verification-and-security/spexplain

[10] In the medical datasets, the training of NNs beyond 8 layers often does not converge.

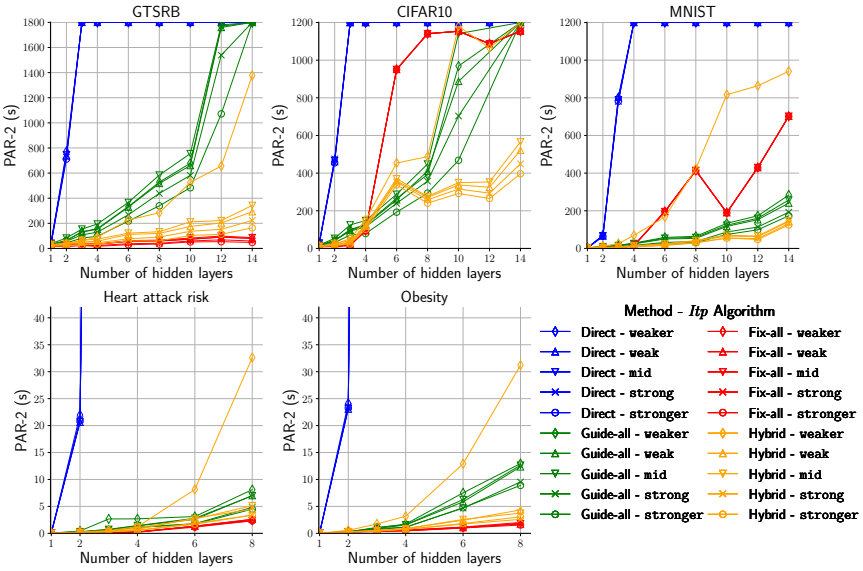


Fig. 2: Performance comparison of our methods and interpolation algorithms. In medical datasets, it shows PAR-2 only up to 40 to make methods other than *Direct* visible.

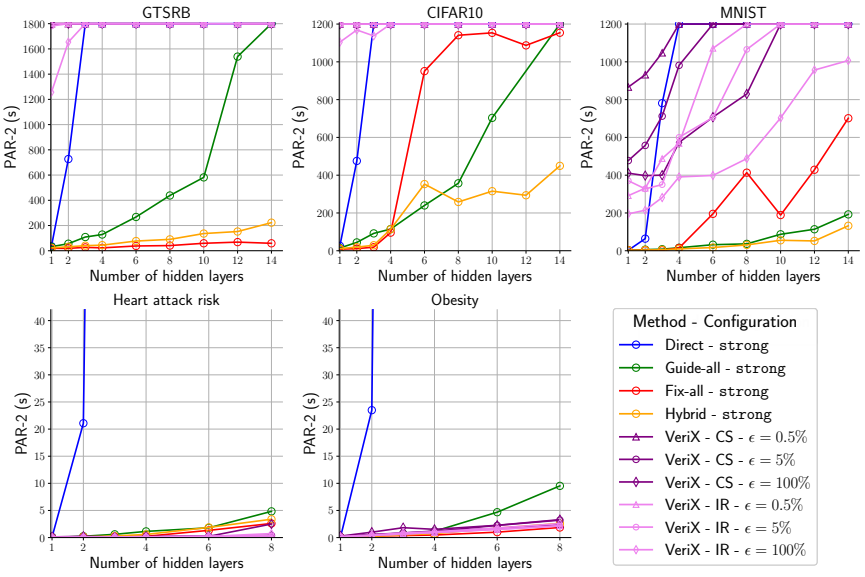


Fig. 3: Performance comparison of `strong` variants with VeriX.

5.1 SMT and Craig interpolation Instantiation

We instantiated the framework from Fig. 1 with the OpenSMT as the **Solver** and used its interpolating features as the procedures for the **Generalizer**. To support decision guidance capability, we implemented Alg. 3 in OpenSMT

(**Solver**). The choice of tools is intentional to support a direct comparison with the experimental study [32], referred to as *Direct*. We compare four different configurations of our method: *Direct*: $\mathcal{G}_{\mathcal{H}} = \mathcal{F}_{\mathcal{H}} = \emptyset$ (the implementation used in [32]); *Guide-all*: $\mathcal{G}_{\mathcal{H}} = \mathcal{V}_{\mathcal{H}}$, $\mathcal{F}_{\mathcal{H}} = \emptyset$; *Fix-all*: $\mathcal{G}_{\mathcal{H}} = \emptyset$, $\mathcal{F}_{\mathcal{H}} = \mathcal{V}_{\mathcal{H}}$; and *Hybrid*: $\mathcal{G}_{\mathcal{H}} = \mathcal{V}_{\mathcal{H}}$, $\mathcal{F}_{\mathcal{H}} = \mathcal{V}_{\mathcal{H}} \setminus \mathcal{R}_{\mathcal{H}}$ (using (3) with adversarial example $\mathbf{s}'$). Instances of interpolation algorithms (cf. Sect. 2) use the notation $Itp_{DF} \mapsto$ `stronger`, $Itp_F \mapsto$ `strong`, $Itp'_F \mapsto$ `weak`, $Itp'_{DF} \mapsto$ `weaker`, and $Itp_f \mapsto$ `mid` with $f := 0.5$. Depending on the complexity of the dataset, the evaluation uses the following computation time limits per individual explanation: GTSRB: 15 min; MNIST and CIFAR10: 10 min; medical datasets: 5 min. To aggregate both the runtime and occasional timeouts, we leverage the metric commonly used in the SMT community, PAR-2, which is the runtime, or in the case of a timeout, twice the timeout. For every NN and explanation technique, we report the average PAR-2 out of all 100 explanations. The experiments were run on a Linux 5.4 machine with 256 GB physical memory and AMD® EPYC 7452 64-threaded CPU.

Fig. 2 shows how different configurations and interpolation algorithms scale to the number of layers, running on particular datasets. The x-axis shows the number of hidden layers, and the y-axis shows the PAR-2 average. The plots show the clear trend that the original *Direct* method always times out when using models with at most 4 layers. In contrast, the new techniques scale substantially better. Notably, even *Guide-all* that encodes the entire network still significantly improves over *Direct*. The *Fix-all* configuration dominates the performance in the medical domain tasks and GTSRB, but experiences timeouts at CIFAR10 and MNIST within deeper networks due to a weakness of the theory solver in OpenSMT (simplex)—it struggles at resolving the overly restricted problem, approaching the worst-case execution. The *Hybrid* configuration addresses this issue and usually achieves the best or second-best performance. The choice of interpolation algorithm has low impact on scaling trends, with one notable exception, *Hybrid* with `weaker`, when it produces many non-convex constraints. Explaining with `stronger` performs best but it usually does not generalize at all. In the following, we select the second-best performing algorithm `strong`.

We compare to the state-of-the-art NN explanation tool VeriX[11], using two configurations [52]: *complete* verification with *sensitivity* traversal (CS), which relaxes the sample point most, and *incomplete* verification with *random* traversal (IR), the most performant option. A further parameter is the perturbation radius ϵ that defines the maximal vicinity of the sample point. We adopted [52] $\epsilon \in \{0.5\%, 5\%, 100\%\}$. The value $\epsilon = 100\%$ is conceptually similar to our approach, as features are allowed to range in entire domain. Fig. 3 compares the results from Fig. 2 (using `strong`) and VeriX. VeriX performs better using IR configuration or with higher values of ϵ. VeriX performs significantly worse in the image-classification domain, because it is sensitive to the number of input features [32], and slightly outperforms all our methods only in the heart-attack risk case. Recall

[11] VeriX+ could not be successfully executed on our evaluation suite, running only on the simpler benchmarks with shallow networks reported in [51]. It would be also interesting to compare with the recent results of [7], which remains future work.

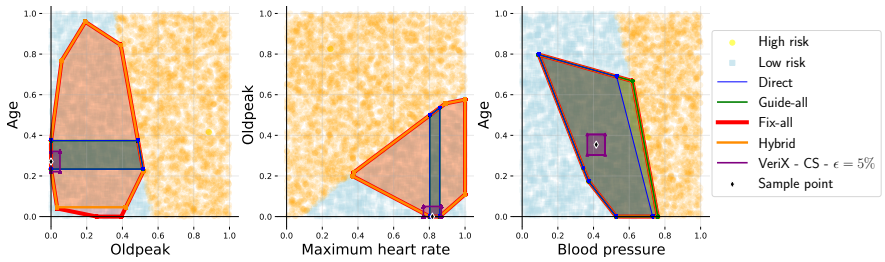


Fig. 4: Visual comparison of selected 2D snapshots of explanation spaces of the heart-attack dataset (with two classes: high and low risk) produced by different methods.

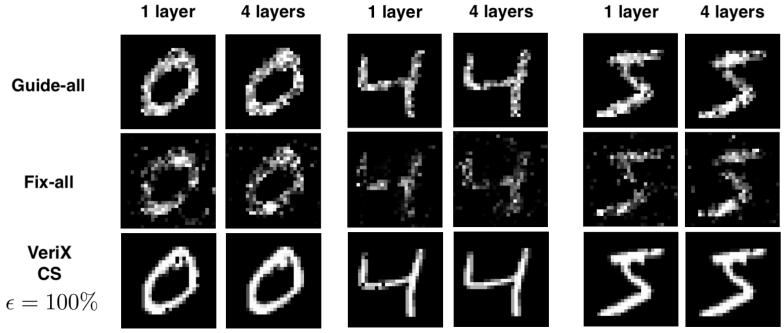


Fig. 5: Visual comparison of generality of explanations produced by different techniques from three MNIST samples of digits 0, 4, and 5. The number of pixels that are darker compared to the original image reflects more general explanations.

that here the usefulness of the explanations produced by VeriX is limited by not capturing relations between features that are important in tabular data. We further experimented with more complex networks (200 neurons per layer). These experiments are much more challenging, with more methods timing out for some network sizes, though key trends from Fig. 3 persist. For the results and discussion, refer to the extended version of this paper [28].

We checked the generality of `strong` explanations[12] computed by our methods using subsumption checks for all pairs of explanations for heart-attack networks having 1 to 4 layers[13]. There is a clear trend in the generality: 98% of explanations produced by *Direct* and *Guide-all* are equivalent, while 100% and 95% of *Guide-all* explanations are subsumed by *Fix-all* and *Hybrid* explanations, respectively, and 98% of *Hybrid* explanations are subsumed by *Fix-all*. If using weaker interpolation algorithms, the checks frequently result in inconclusive cases (explanations intersect but do not subsume each other) due to the higher flexibility in the generalization. While the dominance of *Fix-all* may be counter-intuitive, it reveals that (1) the continuous area restricted by the neuron activations is still large enough for the generalizations, and (2) since this area

[12] Algorithm `strong` focuses on a close neighborhood of the sample point but still often covers spaces with a similar shape compared to weaker algorithms [32].

[13] Beyond 4 layers, *Direct* never finishes, and the subset checks are also very expensive.

is defined by more deterministic, convex constraints, it is easier for the interpolation to generalize the constraints. These results suggest that the generality of explanations is not compromised or even improves when restricting the NN.

The following further supports observations on generality and demonstrates that the explanations remain useful and interpretable. Fig. 4 visually compares selected two-dimensional snapshots of explanations in the feature space [32], using the single-layer heart-attack network and more general algorithm `weak`. Explanations computed by *Fix-all* and *Hybrid* again subsume other explanations, providing the highest generality and also most precise approximation of the decision boundary. *Direct* and *Guide-all* are also general, not restricted to a specific vicinity of the sample points. By contrast, the explanations computed by VeriX (CS, $\epsilon = 5\%$) represent only small rectangles in the feature space.

In image-recognition domain, Fig. 5 demonstrates the generality of `strong` explanations for the MNIST dataset, showing specific points from the feature space (i.e., digit images) that are different from the original sample point but still covered by the explanation. The points are computed using [33], the euclidean minimum norm, and illustrate how much each pixel of the sample can be darkened while still guaranteeing the classification, capturing the flexibility given by the explanation and hence reflecting the size of its impact space. It compares *Guide-all*, *Fix-all*, and VeriX (CS, $\epsilon = 100\%$), confirming that *Fix-all* yields more general explanations than *Guide-all*. Despite the use of `strong` algorithm, both are still substantially more general than VeriX where only a few isolated pixels are switched to black. We also observed that VeriX (CS, $\epsilon = 5\%$) slightly darkens many pixels, but the difference from the original is barely visible. The figure further illustrates that, among all techniques, the explanations remain similar if computed from a NN with one or four layers, suggesting that even restricted encoding of the network (e.g. *Fix-all*) does not harm the generality.

Compared to VeriX, the explanations computed using our technique result in more complex formulas, potentially occupying megabytes rather than kilobytes of memory. However, this does not hinder their interpretability: for instance, computing the minimum norm point in Fig. 5 requires only a few seconds. Further details are provided in the extended version [28].

5.2 Modular Encoding Instantiation

The core intuition behind a modular translation lies in treating a neural network not as a monolithic mathematical function, but as a discrete-time transition system. In this framework, the activation values of neurons in a given layer represent the "state" of the system. The transition from one layer to the next is governed by a recurring logical predicate that encapsulates the linear transformation (weights and biases) and the non-linear activation function ($ReLU$). By defining a generic transition formula, we decouple the architectural definition from the specific instance of the network. This modularity allows formal verification tools to *discover inductive invariants* that describe the properties of

neurons in any hidden layer that can also serve as explanations[14]. The purpose of this section is to demonstrate that this alternative encoding benefits from neuron activation, similarly to when the monolithic encoding is used.

A modular transition ψ_{step} captures the logic of a single layer transition from state $\mathbf{x}^{(\ell)}$ to $\mathbf{x}^{(\ell+1)}$ for any hidden layer $0 \leq \ell < L$:

$$\psi_{step}(\mathbf{x}, \mathbf{x}', \ell, \ell') = 0 \leq \ell < L \wedge \bigwedge_{i=1}^{n} \Big(x'_i = ite\big(\big(\sum_{j=1}^{n} w_{j,i}^{(\ell)} x_j + b_i^{(\ell)}\big) > 0, \sum_{j=1}^{n} w_{j,i}^{(\ell)} x_j + b_i^{(\ell)}, 0\big)\Big) \wedge \ell' = \ell + 1. \tag{4}$$

An inductive invariant Inv is a property of neurons in each hidden layer. Given the initial condition $\varphi_s \wedge \psi_{\mathcal{D}}$ and the error state $\neg\psi_c$, Inv intuitively serves as a symbolic summary of activations that never reach the misclassification region, i.e., the three conditions hold:

1. **Initiation:** $\varphi_s \wedge \psi_{\mathcal{D}} \implies Inv(\mathbf{x}, 0)$
2. **Inductiveness:** $Inv(\mathbf{x}, \ell) \wedge \psi_{step}(\mathbf{x}, \mathbf{x}', \ell, \ell') \implies Inv(\mathbf{x}', \ell')$
3. **Safety:** $Inv(\mathbf{x}, L) \implies \psi_c$

We adopted the winning strategy from Sect. 5.1 *Fix-all* using the precomputed values $A_i^{(\ell)}$ of neuron pre-activations for every layer ℓ and neuron i based on evaluating the neural network on the sample. The modified modular transition ψ'_{step} uses these values to pick only the active or inactive phase:

$$\psi'_{step}(\mathbf{x}, \mathbf{x}', \ell, \ell') = 0 \leq \ell < L \wedge \bigwedge_{i=1}^{n} \Big(x'_i = ite\big(A_i^{(\ell)} > 0, \sum_{j=1}^{n} w_{j,i}^{(\ell)} x_j + b_i^{(\ell)}, 0\big)\Big) \wedge \ell' = \ell + 1. \tag{5}$$

As a proof of concept, we implemented the encoding on top of an existing solver [18,19], which iteratively guesses template instantiations (e.g., as defined below) and checks the initiation and inductiveness conditions. The conjunction of instantiations that passed these checks as well as the safety condition forms a computed explanation. The templates have the following forms: $(\ell = _ \implies x_i^\ell \leq _)$, $(\ell = _ \implies x_i^\ell \geq _)$, $(\ell = _ \implies x_i^\ell = _)$, $(x_i^\ell \leq _)$, $(x_i^\ell \geq _)$, $(x_i^\ell = _)$, where each $_$ is a placeholder. Because the behavior of the invariant synthesis depends heavily on a smart choice of the templates, our implementation does not aim to compete with the monolithic approach. Instead, we evaluate the impact of the fixing strategy by comparing the invariant synthesis runtimes when using ψ_{step} (the *Direct* strategy) and ψ'_{step} (the *Fix-all* strategy). The evaluation follows the setup in Sect. 5.1, using the medical benchmarks[15]. Fig. 6 illustrates a

[14] The main benefit of this non-monolithic encoding is providing better control over the shapes of the computed explanations.

[15] The current prototype invariant synthesis is driven by template selection and is sensitive to the input feature count, timing out on all the image-recognition benchmarks.

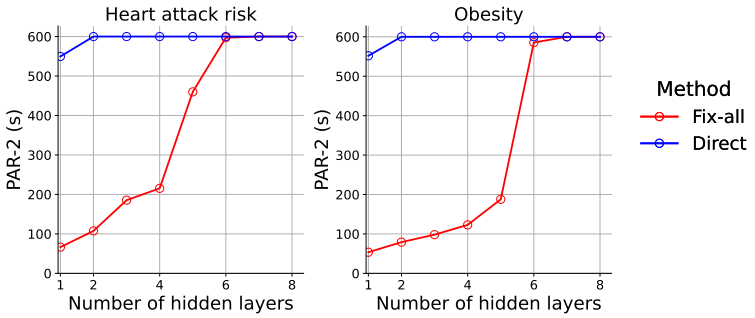


Fig. 6: Performance of an enumerative invariant discovery with/without fixing.

clear advantage of computing explanations using the *Fix-all* strategy (red lines) over the *Direct* strategy (blue lines), which mostly times out. Overall, for both monolithic and non-monolithic encoding of NNs, the data confirms that the fixing is a viable solution to computing logical explanations significantly faster.

6 Conclusion and Future Work

This paper presented a novel neuron activation-based symbolic framework for automated computation of logical explanations that (1) enables scaling to deep architectures not amenable to prior logic-based explainability methods, (2) improves the generality of the resulting explanations that are provably correct and not restricted in shape, as opposed to explanations generated by prior methods that rely on specialized NN verifiers, (3) isolates the NN-specific encoding from the reasoning algorithms that are based on general-purpose logical solvers, instantiated to OpenSMT, utilizing the newly introduced transferable mechanism of guiding constraints, and (4) is agnostic to the backend solver and the techniques that generalize the constraints on the sample point. The experiments on diverse real-world benchmarks from the domains of image recognition (GTSRB, CIFAR10, MNIST) and medicine across a spectrum of complexities showed that the new method computes more general explanations and outperforms prior state-of-the-art approaches, often by orders of magnitude, particularly the leading abduction-based tool for NNs VeriX. The experiments also showed that the approach is generic and applies beyond SMT encoding.

In future work, we will aim to handle other NN structures, such as convolutional NNs, addressing more complex yet efficient classification tasks, with no need for changes in our method at the theoretical level. Moreover, we will aim at embedding other activation functions beyond $ReLU$, such as Leaky $ReLU$.

Acknowledgements. This work was conducted as part of the "Formal Reasoning on Neural Networks" project funded by the Hasler Foundation, Switzerland, and of project 200021-236601 "Cross theory rigorous program verification using Constrained Horn clauses" funded by the Swiss National Science Foundation.

References

1. Albarghouthi, A., Dillig, I., Gurfinkel, A.: Maximal specification synthesis. SIGPLAN Not. **51**(1), 789–801 (Jan 2016), https://doi.org/10.1145/2914770.2837628
2. Alt, L., Hyvärinen, A.E.J., Sharygina, N.: LRA interpolants from no man's land. In: Strichman, O., Tzoref-Brill, R. (eds.) HVC. pp. 195–210. Springer International Publishing, Cham (2017)
3. Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K.R., Samek, W.: On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. PLOS One **10**(7) (2015)
4. Baehrens, D., Schroeter, T., Harmeling, S., Kawanabe, M., Hansen, K., Müller, K.R.: How to Explain Individual Classification Decisions. Journal of Machine Learning Research **11** (2010)
5. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) TACAS. pp. 415–442. Springer International Publishing, Cham (2022)
6. Barrett, C., Fontaine, P., Tinelli, C.: The SMT-LIB Standard: Version 2.6. Tech. rep., Department of Computer Science, The University of Iowa (2017), https://smt-lib.org
7. Bassan, S., Elboher, Y.Y., Ladner, T., Althoff, M., Katz, G.: Explaining, fast and slow: Abstraction and refinement of provable explanations. In: Proceedings of the 42nd International Conference on Machine Learning. Proceedings of Machine Learning Research, PMLR (2025)
8. Blicha, M., Hyvärinen, A.E.J., Kofroň, J., Sharygina, N.: Decomposing Farkas interpolants. In: Vojnar, T., Zhang, L. (eds.) TACAS. pp. 3–20. Springer International Publishing, Cham (2019)
9. Bruttomesso, R., Pek, E., Sharygina, N., Tsitovich, A.: The OpenSMT Solver. In: Esparza, J., Majumdar, R. (eds.) TACAS. LNCS, vol. 6015, pp. 150–153. Springer (2010), https://doi.org/10.1007/978-3-642-12002-2_12
10. Christ, J., Hoenicke, J., Nutz, A.: SMTInterpol: An interpolating SMT solver. In: Donaldson, A.F., Parker, D. (eds.) SPIN. Lecture Notes in Computer Science, vol. 7385, pp. 248–254. Springer (2012), https://doi.org/10.1007/978-3-642-31759-0_19
11. Cimatti, A., Griggio, A., Schaafsma, B., Sebastiani, R.: The MathSAT5 SMT Solver. In: Piterman, N., Smolka, S. (eds.) TACAS. LNCS, vol. 7795, pp. 93–107. Springer (2013)
12. Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. In: J. of Symbolic Logic. pp. 269–285 (1957)
13. De Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: TACAS. pp. 337–340. TACAS'08/ETAPS'08, Springer-Verlag, Berlin, Heidelberg (2008), https://doi.org/10.1007/978-3-540-78800-3_24
14. Deng, L.: The MNIST Database of Handwritten Digit Images for Machine Learning Research. IEEE Signal Processing Magazine **29**(6), 141–142 (2012), https://doi.org/10.1109/MSP.2012.2211477
15. Dillig, I., Dillig, T., Li, B., McMillan, K.L.: Inductive invariant generation via abductive inference. In: Hosking, A.L., Eugster, P.T., Lopes, C.V. (eds.) OOPSLA. pp. 443–456. ACM (2013), https://doi.org/10.1145/2509136.2509511

16. Dutertre, B.: Yices 2.2. In: Biere, A., Bloem, R. (eds.) CAV. pp. 737–744. Springer International Publishing, Cham (2014)
17. Erhan, D., Bengio, Y., Courville, A., Vincent, P.: Visualizing Higher-layer Features of a Deep Network. Tech. rep., University of Montreal (2009)
18. Fedyukovich, G., Bodík, R.: Accelerating Syntax-Guided Invariant Synthesis. In: TACAS, Part I. vol. 10805, pp. 251–269 (2018)
19. Fedyukovich, G., Kaufman, S., Bodík, R.: Sampling Invariants from Frequency Distributions. In: FMCAD. pp. 100–107. IEEE (2017)
20. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. In: Bengio, Y., LeCun, Y. (eds.) ICLR (2015), http://arxiv.org/abs/1412.6572
21. Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: OpenSMT2: An SMT Solver for Multi-core and Cloud Computing. In: Creignou, N., Le Berre, D. (eds.) SAT. pp. 547–553. Springer International Publishing, Cham (2016)
22. Ignatiev, A., Narodytska, N., Marques-Silva, J.: Abduction-based explanations for machine learning models. In: AAAI. AAAI'19/IAAI'19/EAAI'19, AAAI Press (2019), https://doi.org/10.1609/aaai.v33i01.33011511
23. Izza, Y., Ignatiev, A., Stuckey, P.J., Marques-Silva, J.: Delivering inflated explanations. In: AAAI. AAAI'24/IAAI'24/EAAI'24, AAAI Press (2024), https://doi.org/10.1609/aaai.v38i11.29170
24. Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R.: Heart Disease. UCI Machine Learning Repository (1989), https://doi.org/10.24432/C52P4X
25. Kanav, S., Křetínský, J., Rieder, S.: A Literature Review on Verification and Abstraction of Neural Networks Within the Formal Methods Community. Springer Nature Switzerland, Cham (2025), https://doi.org/10.1007/978-3-031-75778-5_3
26. Katz, G., Huang, D.A., Ibeling, D., Julian, K., Lazarus, C., Lim, R., Shah, P., Thakoor, S., Wu, H., Zeljić, A., Dill, D.L., Kochenderfer, M.J., Barrett, C.: The Marabou framework for verification and analysis of deep neural networks. In: Dillig, I., Tasiran, S. (eds.) CAV. pp. 443–452. Springer International Publishing, Cham (2019)
27. Kim, H.: Torchattacks: A pytorch repository for adversarial attacks. arXiv preprint arXiv:2010.01950 (2020)
28. Kolárik, T., Labbaf, F., Leopardi, F., Fedyukovich, G., Wand, M., Sharygina, N.: Neuron activation-based computation of logical explanations for deep neural networks. arXiv preprint (2026), extended version of the paper
29. Kolárik, T., Fedyukovich, G., Labbaf, F., Leopardi, F., Sharygina, N., Wand, M.: Formally explaining neural network classification. In: Sampaio, A., Stoelinga, M. (eds.) Formal Methods. pp. 471–496. Springer Nature Switzerland, Cham (2026)
30. König, M., Bosman, A.W., Hoos, H.H., Van Rijn, J.N.: Critically assessing the state of the art in neural network verification. J. Mach. Learn. Res. **25**(1) (Jan 2024)
31. Krizhevsky, A.: Learning multiple layers of features from tiny images. Tech. rep., University of Toronto (2009)
32. Labbaf, F., Kolárik, T., Blicha, M., Fedyukovich, G., Wand, M., Sharygina, N.: Space explanations of neural network classification. In: Piskac, R., Rakamaric, Z. (eds.) CAV. Springer International Publishing (2025)
33. Leopardi, F., Labbaf, F., Kolárik, T., Wand, M., Sharygina, N.: Interpreting logical explanations of classifying neural networks. In: ESANN (2026)
34. Lundberg, S.M., Lee, S.I.: A Unified Approach to Interpreting Model Predictions. In: NIPS (2017)

35. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: ICLR. OpenReview.net (2018), https://openreview.net/forum?id=rJzIBfZAb
36. Mahendran, A., Vedaldi, A.: Understanding Deep Image Representations by Inverting Them. In: CVPR. pp. 5188 – 5196 (2015)
37. Malfa, E.L., Michelmore, R., Zbrzezny, A.M., Paoletti, N., Kwiatkowska, M.: On guaranteed optimal robust explanations for NLP models. In: IJCAI. pp. 2658 – 2665 (2021)
38. Marques-Silva, J.: Logic-Based Explainability in Machine Learning. Springer Nature Switzerland, Cham (2023), https://doi.org/10.1007/978-3-031-31414-8_2
39. de Moura, L., Bjørner, N.: Satisfiability Modulo Theories: Introduction and Applications. Communications of the ACM **54**(9), 69 – 77 (2011)
40. de Moura, L., Jovanović, D.: A model-constructing satisfiability calculus. In: Giacobazzi, R., Berdine, J., Mastroeni, I. (eds.) Verification, Model Checking, and Abstract Interpretation. pp. 1–12. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
41. Nieuwenhuis, R., Oliveras, A., Tinelli, C.: Solving SAT and SAT modulo theories: from an abstract Davis–Putnam–Logemann-Loveland procedure to DPLL(T). Journal of the ACM (JACM) **53**(6), 937–977 (2006), https://doi.org/10.1145/1217856.1217859
42. Palechor, F.M., Manotas, A.D.l.H.: Estimation of Obesity Levels Based On Eating Habits and Physical Condition. UCI Machine Learning Repository (2019), https://doi.org/10.24432/C5H31Z
43. Prabhu, S., D'Souza, D., Chakraborty, S., Venkatesh, R., Fedyukovich, G.: Weakest precondition inference for non-deterministic linear array programs. In: Finkbeiner, B., Kovács, L. (eds.) TACAS, Part II. Lecture Notes in Computer Science, vol. 14571, pp. 175–195. Springer (2024), https://doi.org/10.1007/978-3-031-57249-4_9
44. Prabhu, S., Fedyukovich, G., D'Souza, D.: Maximal quantified precondition synthesis for linear array loops. In: Weirich, S. (ed.) ESOP, Part II. Lecture Notes in Computer Science, vol. 14577, pp. 245–274. Springer (2024), https://doi.org/10.1007/978-3-031-57267-8_10
45. Ribeiro, M.T., Singh, S., Guestrin, C.: "Why Should I Trust You?": Explaining the predictions of any classifier. In: KDD. p. 1135–1144. KDD '16, Association for Computing Machinery, New York, NY, USA (2016), https://doi.org/10.1145/2939672.2939778
46. Ribeiro, M.T., Singh, S., Guestrin, C.: Anchors: High-Precision Model-Agnostic Explanations. In: AAAI (2018)
47. Seshia, S.A., Sadigh, D., Sastry, S.S.: Towards Verified Artificial Intelligence. Communications of the ACM **65**(7), 46 – 55 (2022)
48. Shih, A., Choi, A., Darwiche, A.: A Symbolic Approach to Explaining Bayesian Network Classifiers. In: IJCAI. pp. 5103 – 5111 (2018)
49. Simonyan, K., Vedaldi, A., Zisserman, A.: Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. In: Workshop Proceedings of the ICLR (2014)
50. Stallkamp, J., Schlipsing, M., Salmen, J., Igel, C.: The German Traffic Sign Recognition Benchmark: A multi-class classification competition. In: IEEE International Joint Conference on Neural Networks. pp. 1453–1460 (2011)

51. Wu, M., Li, X., Wu, H., Barrett, C.: Efficiently computing compact formal explanations. Proceedings of the AAAI Conference on Artificial Intelligence **40**(42), 35857–35866 (Mar 2026), https://doi.org/10.1609/aaai.v40i42.40900
52. Wu, M., Wu, H., Barrett, C.: VeriX: Towards verified explainability of deep neural networks. In: Oh, A., Neumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems. vol. 36, pp. 22247–22268. Curran Associates, Inc. (2023), https://proceedings.neurips.cc/paper_files/paper/2023/file/46907c2ff9fafd618095161d76461842-Paper-Conference.pdf
53. Zeiler, M.D., Fergus, R.: Visualizing and Understanding Convolutional Networks. In: ECCV. pp. 818 – 833 (2014)
54. Zhang, H., Weng, T.W., Chen, P.Y., Hsieh, C.J., Daniel, L.: Efficient neural network robustness certification with general activation functions. In: Advances in Neural Information Processing Systems. vol. 31, pp. 4944–4953 (2018)